Hall Resident Management System

Software Project Lab 3 Mid Technical Report

Submitted by

Abdullah Al Momin
BSSE-1212
Institute of Information Technology
University of Dhaka

Supervised by

Dr. Rezvi Shahariar
Associate Professor
Institute of Information Technology
University of Dhaka

Submission Date: 1 December 2024



Institute of Information Technology
University of Dhaka

Letter of Transmittal

1 December, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission on Hall Management System

Sir,

I am pleased to submit the report titled **"Hall Management System "** as a part of my academic project requirements. This document outlines the design, development, and implementation of the Hall Management System, an innovative platform to streamline residential hall operations at the university.

The report provides a detailed explanation of the project's objectives, methodology, and outcomes, with a focus on key features such as student eligibility determination using machine learning, room allocation, and integrated dashboard management. The system aims to enhance administrative efficiency and improve the user experience for both students and administrators.

I extend my heartfelt gratitude to **Dr**. **Rezvi Shahariar**, whose invaluable guidance and feedback significantly contributed to the success of this project. I would also like to acknowledge the resources and support provided by the Institute of Information Technology throughout the development process.

Thank you for taking the time to review this submission. I look forward to any feedback or questions regarding the report.

Yours sincerely,
Abdullah Al Momin
BSSE 1212

Institute of Information Technology
University of Dhaka

—---

Supervisor's Signature

Acknowledgement

I would like to express my heartfelt gratitude to all those who supported and guided me throughout this project. First and foremost, I would like to thank my supervisor, Rezvi Shahariar, Associate Professor, Institute of Information Technology for his invaluable guidance, support, and insights during the course of this project. Their expertise and feedback were instrumental in shaping the direction of this work.

I would also like to acknowledge the Institute of Information Technology for providing the necessary resources and a conducive research environment. Special thanks to my peers and colleagues, whose discussions and suggestions helped improve this work.

Lastly, I am grateful to my family and friends for their unwavering support, encouragement, and patience during the course of this project.

Abstract

The Hall Management System is a comprehensive solution designed to streamline the administration and management of university residential halls. This system offers a centralized platform to manage key processes such as student applications, room allocations, announcements, and building maintenance. At its core, the system integrates a machine learning-based eligibility determination, which evaluates student applications based on factors like CGPA(or Merit position),distance, academic performance, and financial background, relative living in Dhaka City. The system ensures efficient collaboration between students and administrators while maintaining role-based access to features.

Key functionalities include real-time application status updates, a room allocation module that optimizes building capacity, and an announcement system to keep residents informed. The layered architecture of the system separates the user interface, business logic, and data management layers, ensuring scalability, maintainability. This report provides an in-depth overview of the system's requirements, design, and implementation, highlighting its goal to enhance operational efficiency and improve the residential hall experience for students and administrators alike.

Table Of Contents

Table Of Contents **5**
Chapter 1 ..... **7**
Introduction **7**
1.1 Motivation 7
1.2 Scope 7
1.3 Assumptions 8
1.3 Project Timeline 8
Chapter 2 ..... **9**
Project Description **9**
2.1 Quality Function Development (QFD) 9
2.1.1 Normal Requirements 9
2.1.2 Expected Requirements 9
2.1.3 Exciting Requirements 9
2.2 Usage Scenario 10
Chapter 3 ..... **11**
Scenario-Based Modeling **11**
3.1 Use Case Diagram 11
3.1.1 Use Case Level 0:Hall Management 11
Level 0: High-Level Overview 11
3.1.2 Use Case Level 1:Modules of Hall Management 13
Level 1: Detailed Use Cases 13
Chapter 4 ..... **14**
Data Based Modeling **14**
Chapter 5 ..... **16**
Class-Based Modeling **16**
5.1 Analysis Classes 16
5.2 Class Cards 17
5.2.1 Student 17
5.2.2 Administrator 18
5.2.3 Room 19
5.2.4 Building 20
5.2.5 Application 21
5.2.6 Announcement 22
5.3 CRC Diagram of the classes 23
Chapter 6 ..... **24**
Architectural Design **24**
6.1 Architectural Context Diagram 24

6.2 Defining Archetypes....25
6.3 Top Level Components....26
Chapter 7....27
Preliminary Test Plan....27
7.1 High Level Description of Testing Goals....27
7.2 Test Cases....27
Chapter 8....29
Conclusion....29

Chapter 1

Introduction

University residential halls are integral to fostering a community-centered living and learning environment. Effective hall management ensures students' comfort, safety, and academic focus. However, traditional methods for managing halls often face challenges like manual processing of applications, inefficient room allocation, and a lack of real-time updates.

The **Hall Management System** aims to digitize and streamline hall operations, offering tailored features for both students and administrators. By integrating machine learning, this system introduces intelligent eligibility assessments, efficient room allocation, and comprehensive management capabilities for a modernized hall experience.

1.1 Motivation

Managing university residential halls is a complex task that involves student applications, room allocations, and maintaining real-time communication with residents. Traditional manual processes are time-consuming, error-prone, and inefficient in scaling with the increasing number of students.

The Hall Management System is motivated by the need to modernize hall operations. By leveraging digital tools and machine learning, the system ensures fair eligibility assessments, seamless room allocation, and effective administration. It empowers students and administrators, promoting transparency, efficiency, and enhanced residential life.

1.2 Scope

The Hall Management System caters to two primary user groups:

1. **Students**:
    - Apply for residency.
    - Track application status.
    - Access room allocation details and hall notifications.
    - Use a link to an external meal management system.
2. **Administrators**:
    - Manage applications and verify **eligibility**.
    - Allocate rooms and monitor occupancy.
    - Publish announcements and event notifications.
    - Generate reports on applications, eligibility, and occupancy.

The system supports multi-role authentication, a responsive UI, and integrates public hall details for transparency.

1.3 Assumptions

All users (students and admins) have basic internet access to use the web-based system.
Data such as student details, hall capacities, and application history will be provided or collected during the system's deployment.
Administrators will oversee and verify the training dataset for the ML eligibility model.

1.3 Project Timeline

Table 1: Project Timeline with Tasks

Task	Duration (Weeks)
Literature Review	Weeks 1-2
Project Selection	Weeks 1-2
SRS Documentation	Weeks 3-5
Designing	Weeks 3-5
Development	Weeks 6-14
Testing & Debugging	Weeks 15-17
Final Report Writing	Weeks 18-20
Deployment	Weeks 21-22

Chapter 2

Project Description

2.1 Quality Function Development (QFD)

Quality Function Deployment (QFD) ensures that the user needs and system requirements are effectively translated into the design and development specifications of the Hall Management System. This methodology allows us to prioritize features based on user expectations, focusing on delivering essential functionality while optimizing usability and satisfaction. Below, we detail the **Normal Requirements (NR)**, **Expected Requirements (ER)**, and **Exciting Requirements (ExcR)** for each feature of the system.

2.1.1 Normal Requirements

These are the essential features and functionalities that the tool must have in order to meet the basic requirements for processing and determining resident eligibility.

- Students can apply for hall residency online.
- Machine learning model determines eligibility accurately.
- Administrators can view and process applications.
- Notifications for announcements and application status.

2.1.2 Expected Requirements

- Room allocation is automated and transparent.
- Role-based dashboards for students and admins with visual data insights.
- Administrators can generate reports on hall occupancy and application trends.

2.1.3 Exciting Requirements

- Integration with an external meal system for convenience.
- Responsive, user-friendly web-interface for mobile and desktop users.
- Recommendation for roommate matching

2.2 Usage Scenario

2.2.1 **Scenario 1: Student Application Process**

- A student logs into the system, fills out the residency application, and uploads the required documents.
- The ML model evaluates eligibility based on academic results(CGPA or Merit Position), family income, and distance, Relatives living in Dhaka.
- The student receives real-time feedback on the status of their application (pending, accepted, or rejected).

Scenario 2: Administrator Room Allocation

- The admin logs into the system and views the list of eligible students.
- Rooms are allocated manually or using an automated suggestion feature.
- Occupancy reports are generated to ensure optimal hall utilization.

Scenario 3: Public Information

- Visitors to the public portal view hall information, including buildings and facilities.
- Announcements, such as deadlines or maintenance schedules, are displayed for logged-in students and admins.

Chapter 3

Scenario-Based Modeling

Scenario-based modeling is a technique used in software development and design to create models that illustrate how a system responds to different real-world situations or scenarios. These scenarios are often based on user interactions, events, or specific conditions, and they help developers visualize how the system will behave in various contexts. By simulating these scenarios, developers can identify potential problems, make more informed design decisions, and ensure that the software fulfills user requirements under different conditions.

3.1 Use Case Diagram

A use case diagram is a graphical representation in software engineering that shows how a system interacts with external entities or actors to accomplish specific tasks or objectives. It illustrates the relationships between use cases (which represent system functionalities) and actors (which represent external users or other systems). Use case diagrams are typically created during the requirements gathering phase, where system functionalities and user interactions are defined, and then mapped out using specialized software modeling tools.

3.1.1 Use Case Level 0:Hall Management

Level 0: High-Level Overview

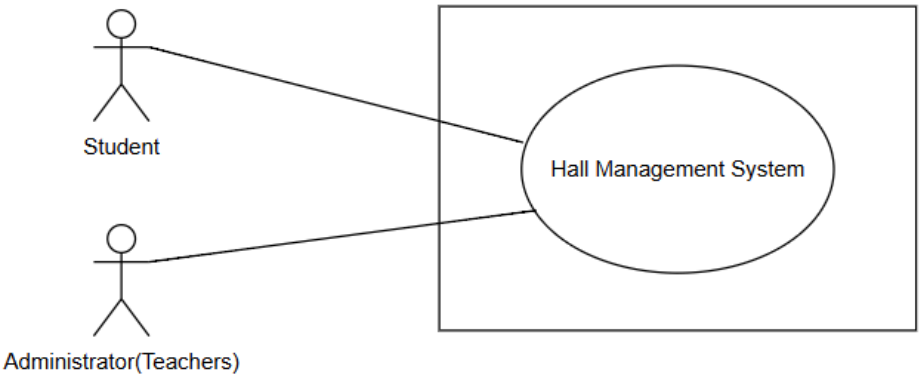


Figure 1: UseCase Level 0

This level outlines the primary interactions between actors (students, administrators) and the system.

- **Actors**:
    - **Student**: Applies for residency, checks application status, accesses announcements.
    - **Administrator**: Reviews applications, allocates rooms, posts announcements.

- **System Modules**:
    - Student Application
    - Eligibility Checking
    - Room Allocation
    - Announcement Management
    - Public Information

Description:

- Students interact with the system for application and updates.
- Administrators handle reviews, room allocation, and announcements.
- Public visitors access general hall details.

3.1.2 Use Case Level 1:Modules of Hall Management

Level 1: Detailed Use Cases

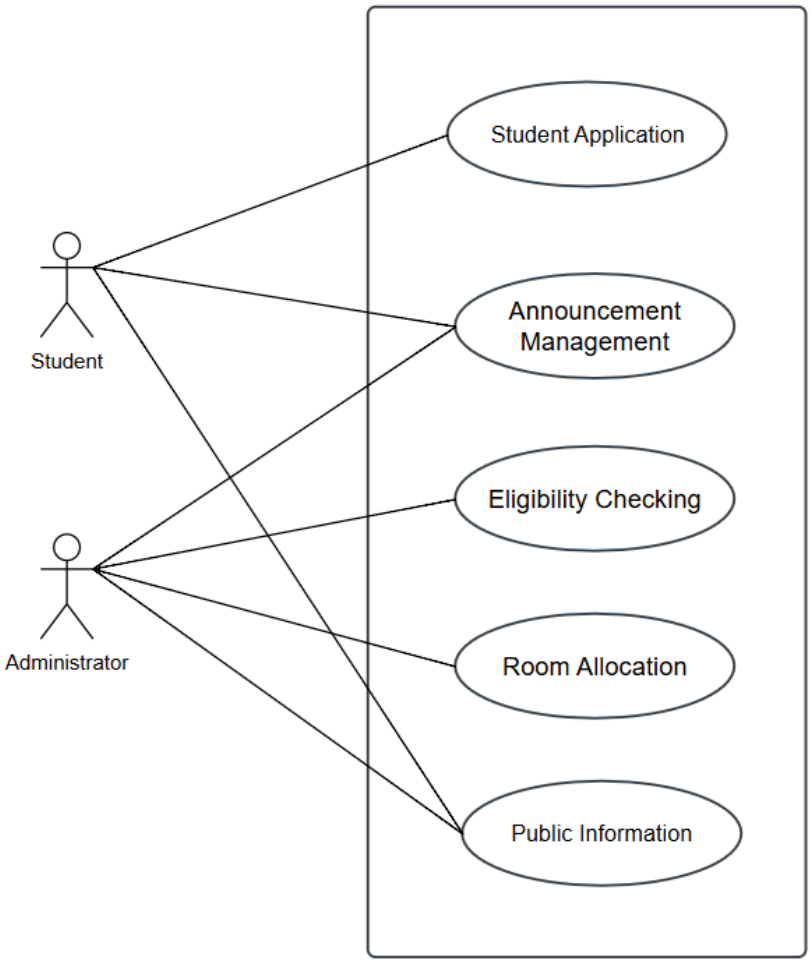


Figure 2: UseCase level 1

At this level, we break down the primary use cases into specific actions.

Use Cases:

1. **Student Application**:
    - Students log in, fill out an application, and upload documents.
    - System checks eligibility and updates status.
2. **Room Allocation**:Admin views eligible students and assigns rooms (manual/automatic).
3. **Announcements**:Admin creates announcements for events, deadlines, or maintenance.
4. **Public Hall Information**:Public users access hall details like facilities and images.

Chapter 4

Data Based Modeling

Database modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

The entities of Recommendation System after analyzing the usage story are:

- Student
- Administrator
- Application
- Room
- Building
- Announcement

Here is the ER-diagram in figure-3 that describes how entities are connected to each other with their relation. In each entity there is a primary key .

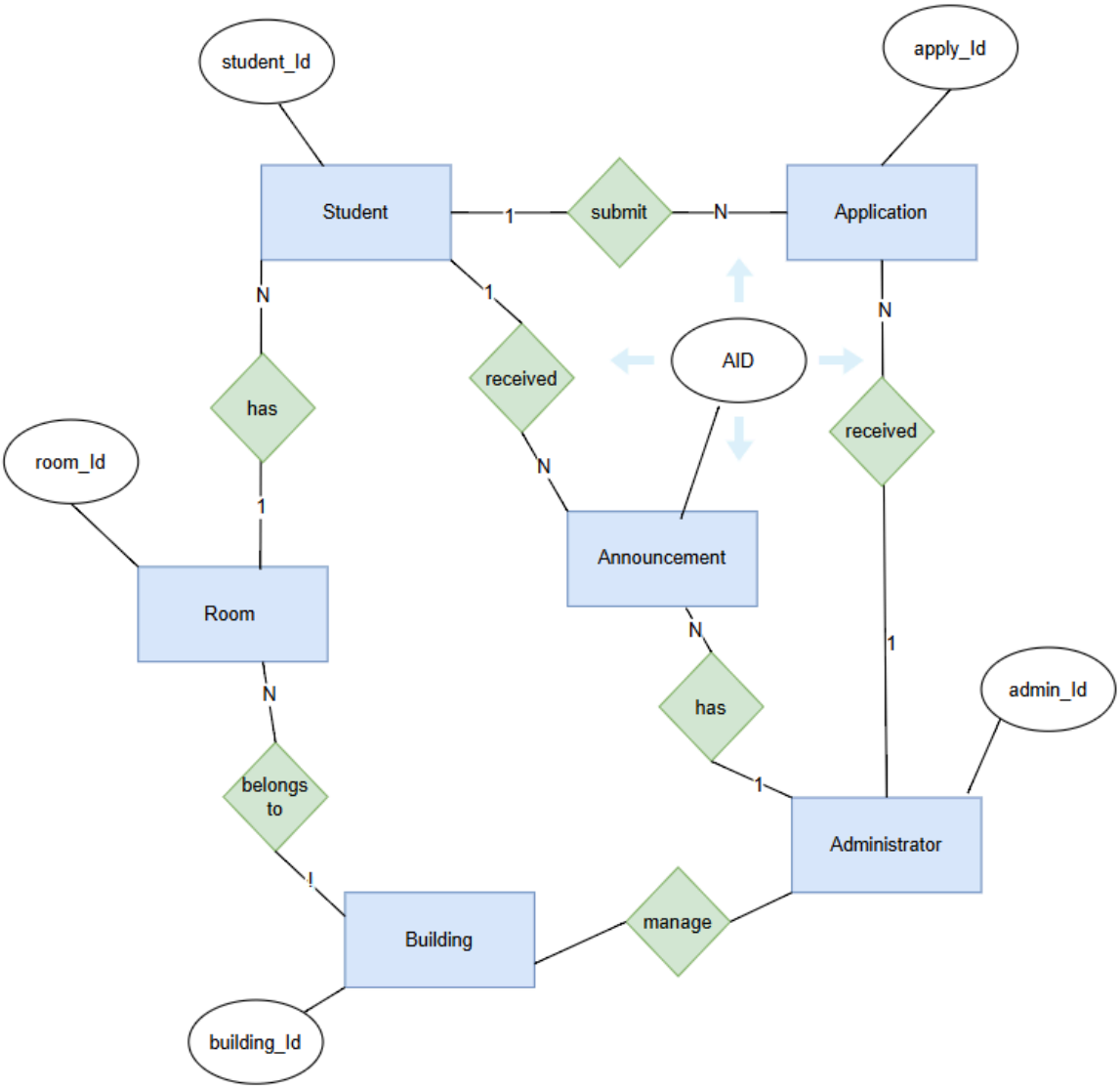


Figure 3:ER Diagram

Chapter 5

Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures).Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes

1. Student
2. Administrator
3. Room
4. MaintenanceRequest
5. Building
6. Application
7. Announcement
8. Notification

Analysis: The class MinatananceRequest has little responsibility, so we can merge it with Application. Again the AreaCalculator class has only one responsibility, that is to calculate area, so we can merge it with ResultHandler. So our finalized classes are:

1. Student
2. Administrator
3. Room
4. Building
5. Application
6. Announcement

5.2 Class Cards

5.2.1 **Student**

Table 2: Class card of Student

Student	
Attributes	**Methods**
-studentId(PK) -name -email -phone -gender -department -year -residentStatus	+applyForHall() +checkApplicationStatus() +viewAnnouncments() +updateProfile
Responsibility	**Collaborator**
● Apply for hall residency. ● Check application status and view announcements. ● Update personal information.	● Application ● Announcement ● Room .

5.2.2 **Administrator**

Table 3: Class card of Administrator

Administrator	
Attributes	**Methods**
-admin_Id(PK) -name -email -role -manageBuildings	+ reviewApplications() + allocateRooms() + postAnnouncement() + generateReports()
Responsibility	**Collaborator**
● Manage and review applications. ● Allocate rooms to students. ● Post announcements and generate reports.	● Application ● Announcement ● Room ● Building.

5.2.3 Room

Table 4: Class card of Room

Room	
Attributes	**Methods**
-roomNumber(P) -building_Id -capacity -occupancyStatus	+ checkAvailability() + assignStudent() + releaseRoom() + getRoomDetails()
Responsibility	**Collaborator**
● Manage room details and availability. ● Handle student assignments to rooms.	● student ● Building.

5.2.4 Building

Table 5: Class card of Room

Building	
Attributes	**Methods**
-building_Id(P) -totalRooms -occupancyStatus	+ checkAvailability() + assignStudent() + releaseRoom() + getRoomDetails()
Responsibility	**Collaborator**
● Manage building information. ● Provide details about halls and facilities.	● Room ● Administrator.

5.2.5 Application

Table 6: Class card of Application

Application	
Attributes	**Methods**
-appication_Id(PK) -student_Id(FK) -applicationDate -status	+ submitApplication() + checkEligibility() + updateApplicationStatus()
Responsibility	**Collaborator**
● Handle student applications for hall residency. ● Evaluate eligibility and update application status.	● Student. ● Administrator.

5.2.6 Announcement

Table 7: Class card of Application

Announcement	
Attributes	**Methods**
-announcement_Id(PK) -title -datePosted	+ createAnnouncement() + editAnnouncement() + deleteAnnouncement() + viewAnnouncement()
Responsibility	**Collaborator**
● Create, edit, and delete announcements. ● Share announcements with relevant audiences.	● Student. ● Administrator.

5.3 CRC Diagram of the classes

Here is in figure-4 the CRC diagram that represents how classes are connected to each other and also shows connection with their appropriate relation name.

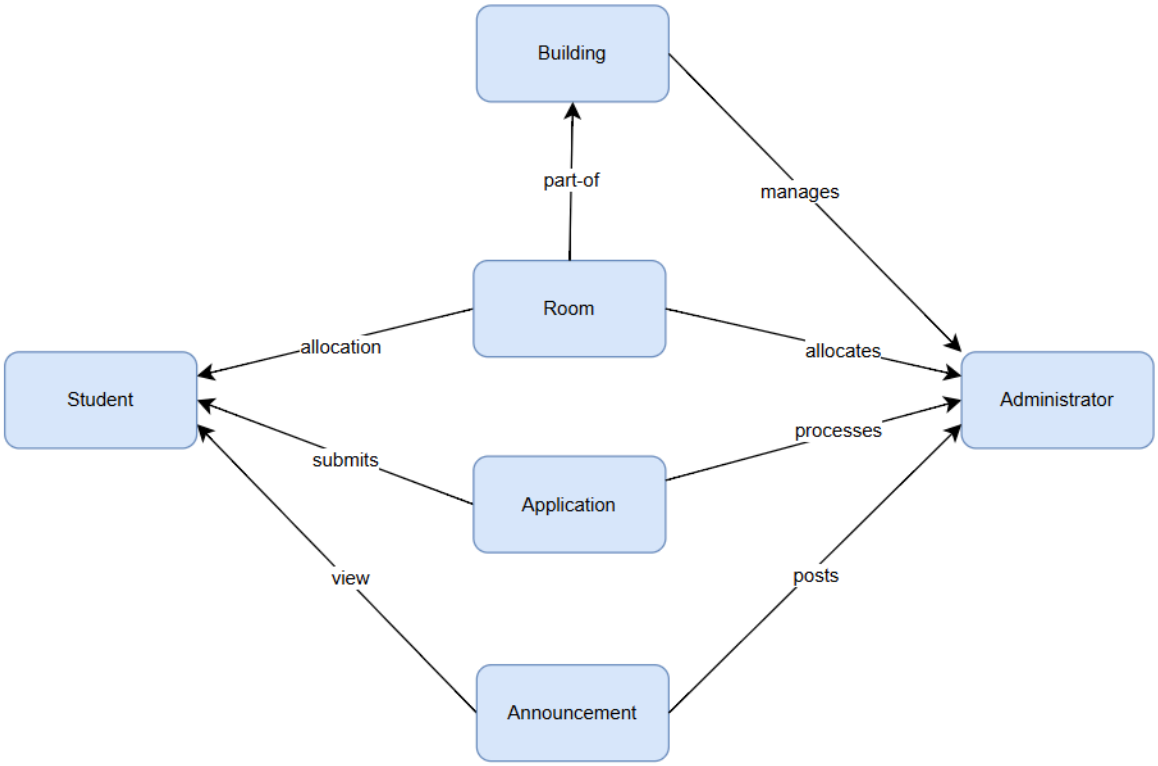


Figure 4: CRC Diagram

Chapter 6

Architectural Design

6.1 Architectural Context Diagram

It is possible to model how software communicates with entities outside of its boundaries using an architectural context diagram. The target system's cooperating systems are denoted by the following symbols:

Superordinate systems: Those systems that use the target system as part of some higher-level processing scheme. The proposed project has no superordinate systems.

Subordinate systems: Those systems that are used by the target system and provide data or processing that are necessary to complete the target system functionality. There are no subordinate systems in our project.

Peer-level systems: Those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system. There are no peer-level systems in our project.

Actors: Entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing. The user of our system is the Actor in this case. Each of these external entities communicates with the target system through an interface (the small shaded rectangles).

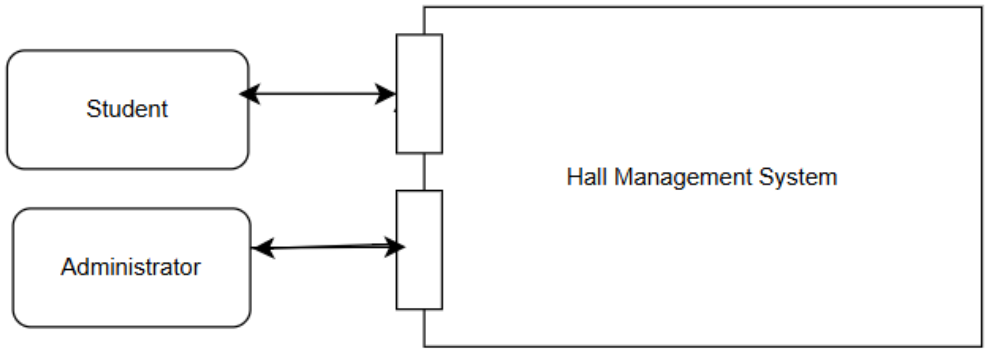


Figure 5:Architectural Context Diagram

6.2 Defining Archetypes

An archetype refers to a class or template representing a fundamental concept crucial for shaping the architecture of the intended system. In essence, even highly intricate systems can be devised by employing a relatively limited set of these archetypes. These archetypes represent the enduring elements of the desired system structure, although they can take on different forms depending on how the system behaves. Together, these archetypes constitute the architecture of the intended system.
Suitable Archetype:Model View Controller (MVC) Architecture Pattern

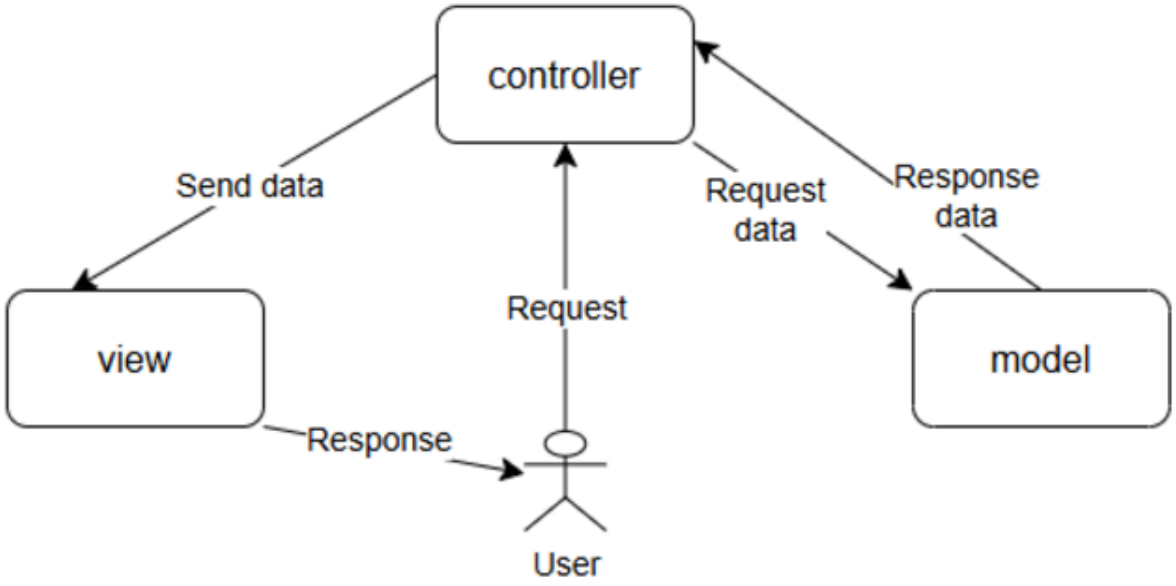


Figure-6: MVC architecture pattern

View: Responsible for presenting data to users and handling user interactions. Display recommendations, analytics, and product details dynamically. Capture user feedback (visit, ratings) and interactions.

Controller: Acts as a bridge between the Model and View, processing user requests and updating the view. Process user actions like product views, feedback submission, and purchases. Call the appropriate model logic and update the UI dynamically.

Model: Responsible for managing the data, logic, and rules of the application. Store and manage user preferences, product details, feedback, and analytics. Handle data-related operations like CRUD, business rules, and database interactions.

6.3 Top Level Components

Overall architectural structure with top level components is illustrated below in figure-8

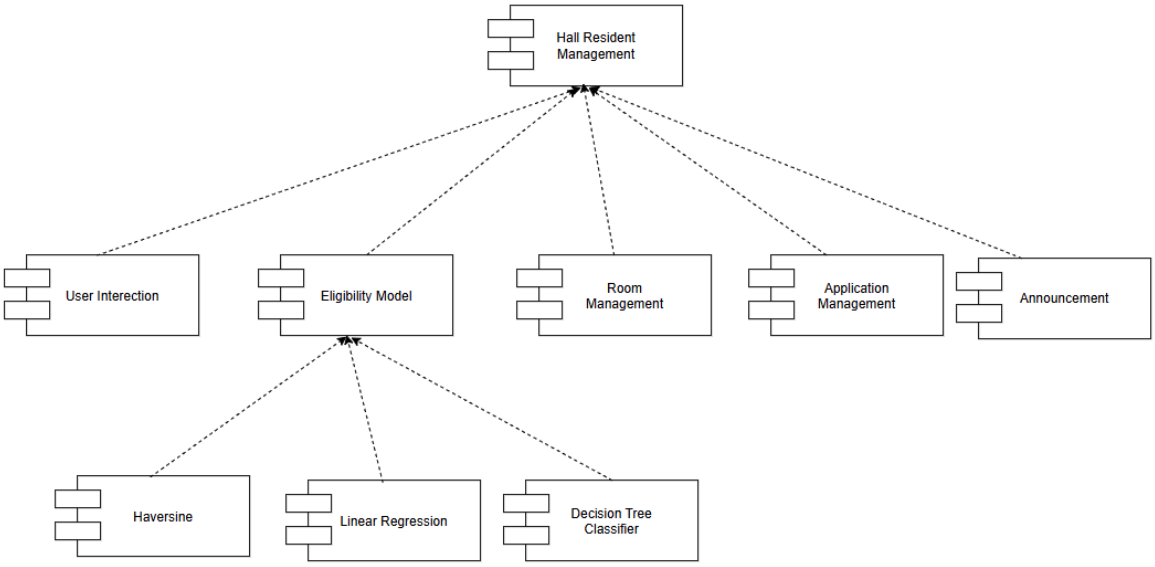


Figure 7: Top Level Architecture

Chapter 7

Preliminary Test Plan

In this chapter, a high-level description of testing goals and a summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

The testing goals for this tool are represented below from a high level:

- To demonstrate that the application meets its requirements
- To ensure that it produces correct detection most of the time
- To ensure that the false positive and negative results are mitigated
- To ensure that it provides a detailed analysis report
- To ensure that the application runs smoothly

7.2 Test Cases

Test cases for the testing of this tool are documented below-

1. **Test Case ID: TC001**
2. Test Case Title: Eligibility Prediction Accuracy
3. Test Scenario: Validate that the system accurately predicts student eligibility based on ML model inputs.
4. Expected Outcome(s):
    - The prediction matches expected results for test data.
    - Error rate is below the acceptable threshold.

1. **Test Case ID: TC002**
2. Test Case Title: Application Submission
3. Test Scenario: Ensure the system allows students to submit applications with complete data.
4. Expected Outcome(s):
    - The application is stored in the database with a status of "Pending".
    - The student receives a confirmation message.

1. **Test Case ID: TC003**
2. Test Case Title: Incomplete Application Submission
3. Test Scenario: Ensure the system gracefully handles missing fields in the application form.
4. Expected Outcome(s):
    - Descriptive error messages indicate missing fields.
    - The user cannot submit the form until all required fields are filled.
    -

1. **Test Case ID: TC004**
2. Test Case Title: Room Allocation Accuracy

3. Test Scenario: Validate that the system allocates rooms correctly based on availability and capacity.
4. Expected Outcome(s):
    - The system assigns an appropriate room to the student.
    - The room's capacity is updated accordingly.

1. **Test Case ID: TC005**
2. Test Case Title: Announcement Notification
3. Test Scenario: Ensure that announcements are visible to the target audience (students/admins).
4. Expected Outcome(s):
    - Students see relevant announcements on their dashboards.
    - Admins can view, edit, or delete announcements as required.

1. **Test Case ID: TC006**
2. Test Case Title: False Positive Mitigation
3. Test Scenario: Validate that the system does not incorrectly approve ineligible students.
4. Expected Outcome(s):
    - Ineligible test cases are correctly rejected.
    - The system provides a detailed explanation of rejection.

1. **Test Case ID: TC007**
2. Test Case Title: Room Overbooking Prevention
3. Test Scenario: Ensure that the system prevents allocating more students to a room than its capacity.
4. Expected Outcome(s):
    - Allocation fails if the room is full.
    - The system provides an appropriate error message to the admin.

1. **Test Case ID: TC008**
2. Test Case Title: Report Generation for Eligibility
3. Test Scenario: Validate that the system generates a detailed report for eligibility decisions.
4. Expected Outcome(s):
    - Reports include input parameters, prediction results, and decision reasoning.
    - Reports are accessible to admins for review.

1. **Test Case ID: TC09**
2. Test Case Title: Performance Under Heavy Load
3. Test Scenario: Ensure the system remains functional and responsive when multiple users access it simultaneously.
4. Expected Outcome(s):
    - System processes all requests without crashing or lagging.
    - Response time remains within acceptable limits.

Chapter 8

Conclusion

The Hall Management System is a robust and efficient solution designed to streamline the administration and management of university residential halls. By integrating essential features such as student application management, room allocation, and announcements, the system ensures that both students and administrators can interact seamlessly and effectively. The incorporation of a machine learning-based eligibility predictor further enhances decision-making, providing a data-driven approach to residential card allocation.

This system adheres to a layered architectural design, ensuring scalability, modularity, and ease of maintenance. It addresses key challenges in hall management, such as real-time updates, error handling, and secure data management, while also incorporating features that improve user experience, such as personalized dashboards and notification systems.

By focusing on core functionality and extending the system with expected and exciting requirements, this Hall Management System is well-positioned to meet the needs of a diverse university community. With its ability to adapt to future requirements, the system represents a significant step toward modernizing residential hall management processes.